Software Detailed Design Document

Projekttitel	Feels Like
Erstellt am	14. Januar 2026
Zuletzt aktualisiert von	Julian Stengele
Zuletzt aktualisiert um	14. März 2026 10:09
Überprüfer/-innen	Atu Isabella Schwarz Julian Stengele Sophie Lazarjan
Erstellt von	Julian Stengele
Kategorie	Strategiedokument

Versionsverwaltung

Version	Datum	Änderungen
V 1.0.0	14. Januar 2026	Initialversion
V 1.1.0	16. Februar 2026	Anpassung des Implementierungsumfang (Entität Wettbewerb gestrichen)
V 1.2.0	05. März 2026	Detaillierte Beschreibung der Module erweitert

▼ 1. Übersicht der Softwarearchitektur

1.1 Architekturmuster

Basierend auf den im Software-Entwicklungsplan (SDP) festgelegten Technologien (React für das Frontend, Node.js/Express für das Backend, SQL für die Datenbank) wählen wir eine klassische **3-Schichten-Architektur (3-Tier-Architecture)**.

Diese Architektur trennt klar zwischen Präsentation, Geschäftslogik und Datenhaltung, was die Wartbarkeit erhöht und paralleles Arbeiten im Team (Frontend/Backend) ermöglicht.

- **Präsentationsschicht (Oberflächenpaket):** React (Single Page Application). Verantwortlich für die Darstellung (UI) und Benutzerinteraktion.
- **Logikschicht (Fachklassenpaket):** Node.js mit Express. Verantwortlich für die API-Bereitstellung, Verarbeitung der Geschäftslogik und Authentifizierung.
- **Datenhaltungsschicht:** SQL-Datenbank (SQLite). Verantwortlich für die persistente Speicherung der Entitäten.

1.2 Paketdiagramm (Struktur der Module)

```
classDiagram

%% ========================
%% SCHICHT 1: OBERFLÄCHENPAKET
%% ========================
```

```
namespace Oberflächenpaket {
  class LoginView
  class TrainingView
  class RankingView
}

%% ========================
%% SCHICHT 2: FACHKLASSENEBENE
%% ========================
namespace Fachklassenpaket {
  class KontoverwaltungService
  class TrainingService
  class AnalyseService
}

%% ========================
%% SCHICHT 3: DATENHALTUNG
%% ========================
namespace Datenhaltung {
  class AthletRepository
  class TrainingRepository
  class SQL_Tabellen
}

%% ========================
%% EXTERNE SYSTEME (E-Mail)
%% ========================
class EmailSystem

%% ========================
%% ABHÄNGIGKEITEN (Layers: oben -> unten)
%% ========================
%% Oberfläche -> Fachklassen
LoginView ..> KontoverwaltungService
TrainingView ..> TrainingService
RankingView ..> AnalyseService

%% Fachklassen -> Datenhaltung
KontoverwaltungService ..> AthletRepository
TrainingService ..> TrainingRepository
AnalyseService ..> TrainingRepository

%% Datenhaltung -> DB
AthletRepository ..> SQL_Tabellen
TrainingRepository ..> SQL_Tabellen
```

```
%% Fachklassen -> externes System
KontoverwaltungService ..> EmailSystem
```

Beschreibung der Pakete:

1. **Oberflächenpaket (Implementiert im Client)**
   - **Views (Komponenten):** Hier befinden sich die Benutzeroberflächen wie `LoginView` (Authentifizierung) und `TrainingView` (Datenerfassung).
   - **API Client:** Eine Hilfsschicht (`api.js`), die die Kommunikation zum Backend kapselt und REST-Anfragen mittels `axios` durchführt.
2. **Fachklassenpaket (Implementiert im Server/Services)**
   - **Business Services:** Hier liegt die eigentliche Logik (`TrainingService`, `AnalyseService`). Sie sind entkoppelt von der HTTP-Schnittstelle.
   - **Controller:** Die technischen Einstiegspunkte (z. B. `authController.js`). Sie nehmen HTTP-Anfragen entgegen, validieren diese und rufen die Services auf.
3. **Datenhaltung (Implementiert im Server/Models)**
   - **Data Access / Models:** Klassen wie `AthletRepository` (Sequelize Models), die das Laden und Speichern aus der Datenbank übernehmen.

▼ 2. Detaillierte Beschreibung der Module

2.1 Mapping der Klassen

Die Entity-Klassen aus dem Grobdesign werden zu Datenbank-Modellen im Backend und Typ-Definitionen im Frontend gemappt.

Analyseklasse (Grobdesign)	Controller (HTTP-Schicht)	Service (Geschäftslogik)	Status
Kontoverwaltung	`authController.js`	`authService.js`	Implementiert (Login/Logout)
AthletenVerwaltung	`userController.js`	--	Teilweise implementiert (nur `profilLaden`)
TrainingsVerwaltung	`trainingController.js`	`trainingService.js`	Vollstandig implementiert (CRUD)
~~WettbewerbsVerwaltung~~	--	--	Gestrichen
AnalyseService (Ranking)	`rankingController.js`	`analyseService.js`	Implementiert
AnalyseService (Statistik Dashboard)	`trainingController.js`	`trainingService.js` (trainingStatistik)	Implementiert

⚠️ **Scope-Anpassung:** Die Entität `Wettbewerb` (UC-03) wurde aus dem Implementierungsumfang gestrichen.
Grund: Reduzierung der Teamgröße von 5 auf 4 Personen während der Entwicklungsphase. Die verbleibenden Ressourcen werden auf die höher priorisierten Use Cases (UC-01, UC-02, UC-04, UC-05, UC-06) fokussiert.

Die Kontrollklassen aus dem Grobdesign werden in **Controller** (HTTP-Handling) und **Services** (Logik) aufgeteilt:

Analyseklasse	Implementierung
Kontoverwaltung	**Controller:** `authController.js` (Routen: /api/auth/login, /api/auth/logout) **Service:** `authService.js` (Authentifizierung: User-Suche, Passwortprüfung über Athlet.checkPassword, JWT-Erstellung)
Trainingsverwaltung	**Controller:** `trainingController.js` (Route: `/api/training`) **Service:** `trainingService.js`
AnalyseService	**Controller:** rankingController.js (Route: `/api/ranking`) **Service:** `analyseService.js` (Strategy Pattern fuer Ranking: DistanceStrategy, FrequencyStrategy, DurationStrategy)

2.2 Datenmodelle

Die Entity-Klassen aus dem Grobdesign werden als Sequelize-Modelle (ORM) implementiert. Die Datenhaltung erfolgt in einer SQLite-Datenbank.

Athlet (Tabelle: `Athlets`)

Attribut	Typ	Pflicht	Besonderheiten
`id`	INTEGER	Ja	Primary Key, auto-increment (von Sequelize)
`name`	STRING	Ja	
`email`	STRING	Ja	unique, wird als E-Mail validiert
`passwortHash`	STRING	Ja	Wird vor dem Speichern per bcrypt gehasht (beforeCreate/beforeUpdate Hook)
`status`	ENUM('eingeladen', 'aktiv', 'inaktiv')	Nein	Default: `'eingeladen'`
`role`	ENUM('athlet', 'trainer')	Nein	Default: `'athlet'`
`createdAt`	DATE	Ja	Automatisch von Sequelize
`updatedAt`	DATE	Ja	Automatisch von Sequelize

Instanzmethode: `checkPassword(enteredPassword)` : Vergleicht ein eingegebenes Klartext-Passwort mit dem gespeicherten Hash via `bcrypt.compare()`.

Hinweis: Im Grobdesign existiert eine separate Klasse `Trainer`. In der Implementierung wird stattdessen das Feld `role` im Athlet-Modell verwendet (`'trainer'` vs. `'athlet'`), um beide Rollen in einem einzigen Modell abzubilden. Dies vereinfacht die Datenhaltung, da sich Trainer und Athlet nur in der Rolle und den Berechtigungen unterscheiden.

Trainingseinheit (Tabelle: `trainings`)

Attribut	Typ	Pflicht	Besonderheiten
`id`	INTEGER	Ja	Primary Key, auto-increment
`datum`	DATEONLY	Ja	Nur Datum, keine Uhrzeit

Attribut	Typ	Pflicht	Besonderheiten
`sportart`	STRING	Ja	Erlaubte Werte: `'Laufen'`, `'Radfahren'`, `'Schwimmen'` (wird vom Service validiert)
`dauer`	INTEGER	Ja	Dauer in Minuten, min: 1
`distanz`	DOUBLE	Nein	Distanz in km, optional
`note`	TEXT	Nein	Freitext-Notiz
`feelsLikeScore`	INTEGER	Ja	Wert 1--10, subjektive Belastungseinschatzung
`athletId`	INTEGER	Ja	Foreign Key → `Athlets.id`
`createdAt`	DATE	Ja	Automatisch von Sequelize
`updatedAt`	DATE	Ja	Automatisch von Sequelize

Beziehungen (Associations)

```
erDiagram
    Athlet ||--o{ Trainingseinheit : "hat viele (hasMany)"
```

Ein Athlet kann beliebig viele Trainingseinheiten haben. Jede Trainingseinheit gehort genau einem Athleten.

2.3 Schnittstellen-Spezifikation (API)

Alle Endpunkte kommunizieren uber JSON. Authentifizierung erfolgt uber einen JWT-Token, der als HttpOnly-Cookie (`token`) gesendet wird.

Authentifizierung (`/api/auth`) -- authController

Methode	Pfad	Auth	Beschreibung
POST	`/api/auth/login`	Nein	Anmelden. Body: `{ email, password }`. Setzt bei Erfolg ein HttpOnly-Cookie `token` mit JWT (gultig 7 Tage). Antwort: `{ message, user: { id, name, email, role }}`
POST	`/api/auth/logout`	Nein	Abmelden. Loscht das `token`-Cookie. Antwort: `{ message }`

User-Verwaltung (`/api/users`) -- userController

Methode	Pfad	Auth	Beschreibung
GET	`/api/users/me`	Ja	Profil des aktuell eingeloggten Users laden (Session Restore). Gibt alle Felder zuruck ausser `passwortHash`.

Trainings-Verwaltung (`/api/training`) -- trainingController -> trainingService

Methode	Pfad	Auth	Beschreibung
GET	`/api/training`	Ja	Alle Trainingseinheiten des eingeloggten Athleten laden, sortiert nach Datum

Methode	Pfad	Auth	Beschreibung
			absteigend. Optional filterbar über Query-Parameter sportart und zeitraum.
POST	`/api/training`	Ja	Neue Trainingseinheit erstellen. Body: `{ datum, sportart, dauer, distanz?, feelsLikeScore, note? }`. Validierung im Service.
PUT	`/api/training/:id`	Ja	Trainingseinheit bearbeiten. Nur der Besitzer darf andern (Autorisierungsprufung).
DELETE	`/api/training/:id`	Ja	Trainingseinheit loschen. Nur der Besitzer darf loschen (Autorisierungsprufung).
GET	`/api/training/stats`	Ja	Liefert aggregierte Kennzahlen (gesamtDistanz, anzahlTrainings, gesamtDauer, durchschnittFeelsLike), optional filterbar über sportart und zeitraum.

Ranking (`/api/ranking`) -- rankingController

Methode	Pfad	Auth	Beschreibung
GET	`/api/ranking`	Ja	Nur für Trainer (sonst 403). Query: sportart (alle|Laufen|Radfahren|Schwimmen), zeitraum (Tage), metrik (distanz|haeufigkeit|dauer). Antwort: { results: [...] }

Fehlerformat: Bei Fehlern antwortet die API mit `{ message: "Fehlerbeschreibung" }` und dem entsprechenden HTTP-Statuscode (400, 401, 403, 404, 500).

2.4 Middleware-Module

authMiddleware

- **Datei:** `server/src/middleware/authMiddleware.js`
- **Aufgabe:** Schützt Routen vor unautorisiertem Zugriff. Wird als Express-Middleware vor geschützten Route-Handlern eingehängt.
- **Ablauf:**
  1. Liest Token aus Authorization: Bearer <token> (bevorzugt) oder alternativ aus HttpOnly-Cookie token.
  2. Verifiziert den Token mit JWT_SECRET.
  3. Bei Erfolg: req.user = { id, role, name }.
  4. Bei Fehler: Cookie wird geloescht und HTTP 401 zurueckgegeben.
- **Eingabe:** HTTP-Request mit Authorization Header (Bearer <token>) oder HttpOnly-Cookie (token)
- **Ausgabe:** `req.user = { id, role, name }` . Bei ungültigem Token: Cookie wird gelöscht und HTTP 401 zurückgegeben.

2.5 Geschäftslogik: TrainingService

Datei: `server/src/services/trainingService.js`

Implementiert die Kontrollklasse `TrainingsVerwaltung` aus dem Grobdesign als Service-Schicht (Schicht 2 / Fachklassenpaket).

Exportierte Funktionen:

Funktion	Parameter	Ruckgabe	Beschreibung
`trainingErfassen(athletId, daten)`	athletId: number, daten: Object	Promise<Trainingseinheit>	Validiert Eingabedaten, erstellt neues Training
`alleTrainings(athletId, filter)`	athletId: number, filter: { sportart?: string, zeitraum?: string }	Promise<Trainingseinheit[]>	Lädt alle Trainings eines Athleten (nach Datum DESC), unterstützt Filter
`trainingAendern(trainingId, athletId, neueDaten)`	trainingId: number, athletId: number, neueDaten: Object	Promise<Trainingseinheit>	Autorisierung + Validierung + Update
`trainingLoeschen(trainingId, athletId)`	trainingId: number, athletId: number	Promise<void>	Autorisierung + Loschung
`validateTrainingsDaten(daten)`	daten: Object	{ valid: boolean, errors: string[] }	Validiert Pflichtfelder, Wertebereiche, erlaubte Sportarten
`trainingStatistik(athletId, filter)`	athletId: number, filter: { sportart?: string, zeitraum?: string }	Promise<{ gesamtDistanz, anzahlTrainings, gesamtDauer, durchschnittFeelsLike }>	Aggregiert Trainingskennzahlen für das Dashboard.

Validierungsregeln:

- `datum`: Pflichtfeld
- `sportart`: Pflichtfeld, muss `'Laufen'`, `'Radfahren'` oder `'Schwimmen'` sein
- `dauer`: Pflichtfeld, muss > 0 sein
- `distanz`: Optional, falls angegeben >= 0
- `feelsLikeScore`: Pflichtfeld, ganze Zahl zwischen 1 und 10

Autorisierung: Bei `trainingAendern` und `trainingLoeschen` wird gepruft, ob `training.athletId === athletId`. Falls nicht: HTTP 403.

Filterlogik: Der Zeitraumfilter unterstützt die Werte woche, monat, quartal und jahr. Die Funktion buildWhereClause kombiniert Athlet, Sportart und Zeitraum in einer gemeinsamen WHERE-Bedingung.

2.6 Frontend-Komponenten (Oberflächenpaket)

Analyse-View (SDD)	Implementierte Komponente	Datei	Status
LoginView	`Login.jsx`	`client/src/pages/Login.jsx`	Implementiert
TrainingView	`TrainingForm.jsx`	`client/src/pages/TrainingForm.jsx`	Implementiert (Liste + Formular + Loschen)
RankingView	`Ranking.jsx`	`client/src/pages/Ranking.jsx`	Implementiert (Live-Daten aus /api/ranking, Filter: Sportart/Zeitraum/Metrik, Trainer-only Fehlerbehandlung)
DashboardView	`Dashboard.jsx`	`client/src/pages/Dashboard.jsx`	Implementiert (Live-Daten aus /api/training und /api/training/stats, Filter, Kennzahlen und Aktivitaetenliste)
AdminView	`AdminUsers.jsx`	`client/src/pages/AdminUsers.jsx`	Platzhalter

Zusatzliche Infrastruktur-Komponenten:

- `AuthContext.jsx` -- React Context fur globalen User-State, Login/Logout-Funktionen, Session Restore
- `ProtectedRoute.jsx` -- Wrapper-Komponente, die unautorisierte Nutzer auf `/login` weiterleitet
- `Layout.jsx` -- Sidebar-Navigation mit Outlet fur geschutzte Seiten
- `axios.js` -- Zentrale Axios-Instanz mit `baseURL` und `withCredentials: true`

▼ 3. Technische Spezifikationen

3.1 Framework-Entscheidungen

- **Kommunikation:** REST API (JSON Format).
- **Authentifizierung:** Erfolgt über JWT. Primär wird ein HttpOnly-Cookie (token) für Browser-Sessions verwendet. Zusätzlich wird für API-Clients ein Authorization Header im Format Bearer <token> unterstützt.
- **Fehlerbehandlung:** Exceptions werden in der Controller-Schicht abgefangen und in standardisierte HTTP-Statuscodes (500, 400, 404) umgewandelt.

3.2 Verwendete Bibliotheken & Tools

Um die Architektur umzusetzen, werden folgende spezifische Bibliotheken genutzt:

Frontend (Client):

- `vite`: Build-Tool und Entwicklungsserver.
- `react-router-dom`: Für das client-seitige Routing (SPA-Navigation).
- `axios`: HTTP-Client für die Kommunikation mit der REST-API.

Backend (Server):

- `express`: Webserver-Framework.

- `sequelize`: ORM zur Abstraktion der SQL-Befehle.
- `sqlite3`: Datenbank-Treiber für SQLite.
- `bcrypt`: Sicheres Hashing von Passwörtern.
- `dotenv`: Verwaltung von Umgebungsvariablen und Secrets.

▼ 4. Qualitäts- und Sicherheitsaspekte

4.1 Sicherheitsmaßnahmen

- **Passwort-Sicherheit:** Passwörter werden niemals im Klartext gespeichert. Beim Anlegen oder Ändern eines Users wird das Passwort automatisch über Sequelize Lifecycle Hooks (`beforeCreate`, `beforeUpdate`) mittels `bcrypt` gehasht und gesalzen.
- **Authentifizierung:** Wir verwenden **HTTP-Only Cookies** für die Übertragung von JWTs (JSON Web Tokens). Dies verhindert, dass client-seitiges JavaScript (z.B. bei XSS-Angriffen) auf den Token zugreifen kann.
- **Secrets Management:** Sensitive Daten wie `JWT_SECRET` und Datenbank-Zugangsdaten werden nicht im Code hardcodiert. Sie werden über `.env` Dateien geladen, die per .gitignore strikt von der Versionsverwaltung ausgeschlossen sind.
- **Route Protection:** Im Backend prüft authMiddleware bei geschützten Requests die Gültigkeit des Tokens. Dabei werden sowohl Authorization Header (Bearer <token>) als auch das HttpOnly-Cookie (token) unterstützt. Im Frontend verhindert ProtectedRoute, dass nicht eingeloggte Nutzer auf interne Seiten gelangen.

4.2 Entwicklungsprozess

- **Version Control:** Nutzung von Git mit Feature-Branches.
- **Separation of Concerns:** Strikte Trennung von Routing, Controller und Service-Logik zur Gewährleistung der Testbarkeit und Wartbarkeit.

4.3 Entwurfsmuster (Design Patterns)

Zur Sicherstellung einer sauberen Architektur und Wartbarkeit werden im Projekt zwei zentrale Design Patterns eingesetzt:

1. Singleton Pattern (Bereich: Infrastruktur)

- **Kontext:** Die Anwendung benötigt Zugriff auf geteilte Ressourcen, die nur einmal existieren dürfen (Datenbankverbindung, Logging-Instanz).
- **Problem:** Das mehrfache Öffnen von Datenbankverbindungen bei jedem Request würde zu Performance-Problemen und Inkonsistenzen führen.
- **Lösung:** Wir implementieren die Datenbank-Instanz (`sequelize`) und den API-Client (`axios` im Frontend) als Singletons. In Node.js nutzen wir dazu das Modul-System (Caching), das sicherstellt, dass beim Importieren der Datei `db.js` immer dieselbe Instanz zurückgegeben wird.

2. Strategy Pattern (Bereich: Geschäftslogik / Ranking)

- **Kontext:** Die Anwendung muss verschiedene Rankings berechnen (z. B. "Meiste Kilometer", "Meiste Trainingseinheiten", "Längste Dauer").
- **Problem:** Ein Controller, der all diese Varianten mit `if (type == 'km') ... else if (type == 'time') ...` abhandelt, wird schnell unübersichtlich und schwer zu testen (Verstoß gegen das Open/Closed Principle).
- **Lösung:** Wir kapseln die Berechnungslogik in austauschbare Strategien.
  - Der `RankingController` definiert nur den Input (die Trainingsdaten).
  - Es gibt separate Strategie-Funktionen (z. B. `DistanceStrategy`, `FrequencyStrategy`).
  - Zur Laufzeit wird basierend auf dem User-Filter die passende Strategie gewählt, ohne den Controller-Code ändern zu müssen.

▼ 5. Design-Abweichungen vom Grobdesign

Grobdesign	Implementierung	Begründung
Separate `Trainer`-Klasse	`role`-Feld im `Athlet`-Modell	Trainer und Athlet teilen dieselben Stammdaten, Unterscheidung über Rolle ausreichend
`feelsLikeScore` nicht im Grobdesign	Neues Feld in Trainingseinheit	Kern-Feature der Anwendung, subjektive Belastungsbewertung (1--10)
`Wettbewerb`-Klasse	Noch nicht implementiert	Priorisierung: Training-CRUD zuerst
`StatistikFilter`, `RankingFilter`, `CSVDatei`	Noch nicht implementiert	Folgt in späteren Sprints
`AthletenVerwaltung.einladungVersenden()`	Noch nicht implementiert	E-Mail-System noch nicht angebunden
`note`-Feld in Trainingseinheit	Neues optionales Feld	Erweiterung fur Freitext-Notizen zu Trainings